

ROZDZIAŁ 13: INGRESS Z TLS — ROUTING HTTPS PO DOMENIE

Co to jest

Ingress to obiekt Kubernetes który definiuje reguły routingu ruchu HTTP/HTTPS do serwisów wewnątrz klastra. Ingress Controller (w K3s jest to Traefik) czyta te reguły i odpowiednio kieruje ruch. TLS (Transport Layer Security) to protokół szyfrowania — następca SSL. Dzięki niemu połączenie między przeglądarką a serwerem jest zaszyfrowane.

Analogia: Recepcja w hotelu z sejfem

Bez Ingress to jak hotel gdzie każdy pokój ma osobne wejście z ulicy na innym numerze (porcie). Z Ingressem jest jedno główne wejście (port 443) z recepcją (Traefik). Gość pyta "chcę do sali konferencyjnej A" — recepcja (Ingress) sprawdza reguły i kieruje do właściwego pokoju (serwisu). TLS to sejf przez który przechodzi cała komunikacja — nikt postronny nie wie co mówisz.

Co zrobimy

- Wygenerowaliśmy self-signed certyfikat TLS przez openssl z SAN (Subject Alternative Name) dla domeny demo.sre-lab.local
- Zapisaliśmy certyfikat i klucz jako Secret Kubernetes typu `kubernetes.io/tls`
- Stworzyliśmy Ingress resource z regułą: zadania na demo.sre-lab.local → serwis aras-demo-app port 8080
- Dodaliśmy wpis w /etc/hosts na Fedorze: 192.168.122.10 demo.sre-lab.local
- Weryfikacja w przeglądarce: X-Forwarded-Proto: https, X-Forwarded-Server: traefik

Różnica między LoadBalancer Service a Ingress

Przed Ingressem aplikacja była dostępna przez LoadBalancer Service na porcie 8080. Każdy serwis potrzebował własnego LoadBalancer'a (i własnego IP lub portu). Ingress to jeden punkt wejścia dla wielu serwisów — rozróżniani po domenie lub ścieżce URL.

```
PRZED: http://192.168.122.10:8080 → LoadBalancer → aras-demo-app
PO:    https://demo.sre-lab.local → Traefik Ingress → aras-demo-app
       https://api.sre-lab.local  → Traefik Ingress → inny-serwis
```

TLS — dlaczego self-signed nie wystarczy na produkcji

Self-signed certyfikat generujemy sami — przeglądarka nie ufa nam jako urzędowi certyfikacji i pokazuje ostrzeżenie. Na produkcji używa się certyfikatów od zaufanych urzędów (CA — Certificate Authority). Let's Encrypt robi to za darmo i automatycznie. W Kubernetes popularnym rozwiązaniem jest cert-manager który automatycznie pobiera i odnawia certyfikaty Let's Encrypt.

Zastosowania produkcyjne

- Jeden punkt wejścia HTTPS dla wszystkich mikroserwisów w klastrze
- Routing po ścieżce URL: `/api/*` → serwis API, `/static/*` → CDN, `/` → frontend
- Canary deployment: 10% ruchu do nowej wersji, 90% do starej — przez jednego Ingress
- Rate limiting, autentykacja, WAF (Web Application Firewall) — Traefik ma te funkcje wbudowane

Silne strony

- Jeden LoadBalancer dla wszystkich serwisów (oszczędność kosztów w chmurze)
- Centralne zarządzanie certyfikatami TLS
- Routing L7 — po domenie, ścieżce, nagłówkach
- Traefik w K3s — zero dodatkowej instalacji

Slabe strony / typowe problemy

- Ingress Controller to single point of failure — jeśli Traefik padnie, wszystkie serwisy są niedostępne
- Deprecated annotation `kubernetes.io/ingress.class` — należy używać `spec.ingressClassName`
- Self-signed certyfikat powoduje ostrzeżenia przeglądarki
- TLS termination na Ingressie — ruch wewnątrz klastra jest niezaszyfrowany (wymaga mTLS/service mesh)

ROZDZIAŁ 14: HPA — AUTOMATYCZNE SKALOWANIE PODOW

Co to jest

HPA (Horizontal Pod Autoscaler) to kontroler Kubernetes który automatycznie zmienia liczbę replik Deployment na podstawie metryk. "Horizontal" oznacza dodawanie/usuwanie Podów (w odróżnieniu od "Vertical" — zwiększanie zasobów istniejących Podów). HPA co 15 sekund odpytuje metrics-server o aktualne zużycie CPU/RAM przez Pody i porównuje z konfigurowanymi progami.

Analogia: Automatyczny dobor kasjerek w supermarkecie

Wyobraz sobie supermarket który ma od 2 do 8 stanowisk kasowych. O poranku jest mały ruch — działają 2 kasy. Gdy przychodzi lunch i kolejki się wydłużają (CPU > 50%), kierownik automatycznie otwiera kolejne stanowiska aż do 8. Po szczycie zamyka zbędne. HPA to ten automatyczny kierownik — reaguje na obciążenie bez ludzkiej interwencji.

Co zrobiliśmy i wyniki

- HPA skonfigurowany: minReplicas: 2, maxReplicas: 8, targetCPU: 50%
- Problem: HPA pokazywał cpu: unknown — brak `resources.requests` w Deployment
- Poprawka: dodano requests cpu: 10m, memory: 32Mi — po rolling update HPA zaczął działać
- Test obciążeniowy: `ab -n 50000 -c 50` — 1621 req/s przez 30 sekund
- Scale-up: CPU 860% → Kubernetes dodał repliki do 8 (maksimum)
- Scale-down: po 5 minutach bez ruchu → powrót do 2 replik (cooldown period)

Dlaczego CPU 860%? Wartość jest relatywna do requests (10m). Jeśli Pod zużywa 86m CPU a ma requests 10m, HPA widzi to jako 860%. To nie jest błąd — to zamierzone zachowanie, `resources.requests` to "minimalna gwarantowana" ilość, nie maksymalna.

Jak HPA oblicza liczbę replik

```
pozadana_liczba_replik = ceil(aktualne_repliki * (aktualne_CPU / docelowe_CPU))  
Przykład: 3 repliki, CPU 860%, cel 50%  
= ceil(3 * (860 / 50)) = ceil(51.6) = 8 (ograniczone przez maxReplicas)
```

Cooldown period — dlaczego scale-down jest powolny

HPA czeka domyslnie 5 minut przed skalowaniem w dol (--horizontal-pod-autoscaler-downscale-stabilization). To zabezpieczenie przed "flappingiem" — sytuacja gdzie HPA ciagle dodaje i usuwa Pody bo obciazenie oscyluje wokol progu. W naszym tescie widzielismy to wyraznie: po zakonczeniu testu przez 5 minut bylo 8 replik, potem HPA sie zdecydowal na scale-down do 2.

Zastosowania produkcyjne

- E-commerce: przed Black Friday ustawiasz maxReplicas: 50, w normalny dzien min: 3
- API servers: automatyczne reagowanie na spike ruchu bez budzenia on-call inzyniera
- Batch jobs: skalowanie w gore przy duzej kolejce zadan, w dol gdy kolejka pusta (KEDA)
- Oszczednosc kosztow w chmurze: placi sie za faktycznie uzywane zasoby

Silne strony

- Automatyczna odpowiedz na obciazenie bez ingerencji operatora
- Oszczednosc kosztow — usuwanie zbednych Podow gdy ruch spada
- Wbudowane w Kubernetes — zero dodatkowej instalacji
- Obsluguje wiele metryk: CPU, RAM, metryki custom przez Prometheus Adapter

Slabe strony / typowe problemy

- Wymaga resources.requests w Deployment — bez tego cpu: unknown (nasz ISSUE-010)
- Nie dziala dla StatefulSets bez dodatkowej konfiguracji
- Cooldown 5 min moze byc za wolny dla aplikacji z naglymi spikami
- Skalowanie Podow jest reaktywne nie predykcyjne — dodaje repliki juz po wzroscie CPU

ROZDZIAŁ 15: NETWORKPOLICY — FIREWALL WEWNATRZ KLASTRA

Co to jest

NetworkPolicy to obiekt Kubernetes definiujący reguły ruchu sieciowego między Podami. Domyslnie Kubernetes używa modelu "wszystko dozwolone" — każdy Pod może łączyć się z każdym. NetworkPolicy zmienia to na model "deny-all, allow explicite" — blokujesz wszystko, potem otwierasz tylko niezbędne połączenia.

Analogia: Kontrola dostępu w biurówcu

Wyobraź sobie biurowiec bez żadnej kontroli dostępu — każdy może wejść do każdego pokoju. NetworkPolicy to karta dostępu: recepcjonistka (Kubernetes) sprawdza przy każdym drzwiach czy masz uprawnienia. Karta księgowego otwiera pokój księgowości i wspólna kuchnia — ale nie serwerownia ani gabinet prezesa.

Co zrobimy

- Stworzylimy NetworkPolicy dla Podów z etykietą `app=aras-demo-app`
- Domyslnie: BLOKUJ cały ruch przychodzący (Ingress)
- Wyjątki dozwolone: kube-system (Traefik), monitoring (Prometheus), repliki aras-demo-app
- Test 1: nieautoryzowany Pod z curl → timeout/blokada → "ZABLOKOWANY - NetworkPolicy działa"
- Test 2: Traefik (kube-system) → odpowiedź aplikacji → dozwolony

Jak działa implementacja (CNI)

NetworkPolicy to tylko definicja w Kubernetes API — sama w sobie nic nie robi. Wykonanie polityk jest odpowiedzialnością wtyczki sieciowej (CNI — Container Network Interface). K3s używa Flannel z VXLAN który obsługuje podstawowe NetworkPolicy. W produkcji częściej używa się Calico lub Cilium — oferują zaawansowane polityki L7 (po protokole HTTP, gRPC) i lepszy performance.

Zastosowania produkcyjne

- Separacja namespace: namespace "frontend" nie może łączyć się z "database" bezpośrednio — tylko przez "backend"
- Multi-tenant klastre: firmy A i B na tym samym klastrze, ich Pody nie mogą się "widzieć"
- Compliance: PCI-DSS wymaga izolacji systemów płatności — NetworkPolicy to implementuje
- Defense in depth: nawet jeżeli atakujący przejmie jeden Pod, nie może skakać po klastrze

Silne strony

- Zasada least privilege dla ruchu sieciowego
- Deklaratywna — polityki są w YAML, wersjonowane w Git
- Granularna kontrola: po namespace, etykietach Podów, portach
- Wbudowana w K8s API — działa z każdym CNI który ją obsługuje

Slabe strony / typowe problemy

- Domyślny CNI (Flannel) ma ograniczone wsparcie — Calico/Cilium do zaawansowanych scenariuszy
- Trudny debugging — gdy coś nie działa ciężko stwierdzić czy to polityka czy konfiguracja sieciowa
- Brak globalnych polityk (ClusterNetworkPolicy) w standardowym K8s
- Polityki są addytywne — dwie polityki = unia regul (nie można jedna nadpisać drugiej)

ROZDZIAŁ 16: RBAC — KONTROLA DOSTĘPU OPARTA NA ROLACH

Co to jest

RBAC (Role-Based Access Control) to system uprawnień w Kubernetes. Definiuje kto (Subject: użytkownik, grupa, ServiceAccount) może wykonać co (Verbs: get, list, create, delete) na czym (Resources: pods, deployments, secrets). Zasada least privilege: każdy podmiot ma dokładnie tyle uprawnień ile potrzebuje — ani trochę więcej.

Analogia: Uprawnienia pracowników w firmie

Pracownik magazynu może przyjmować i wydawać towary (get, create na "shipments") ale nie może zmieniać cen (update na "prices") ani zwalniać pracowników (delete na "employees"). Jeśli jego konto zostanie przejęte przez hakera, szkody są ograniczone do magazynu. Konto administratora z pełnymi uprawnieniami dałoby hackerowi władzę nad całą firmą.

Cztery komponenty RBAC

- **ServiceAccount** — konto techniczne dla aplikacji (nie dla ludzi). Każda aplikacja powinna mieć dedykowany SA zamiast używać wspólnego "default"
- **Role** — zestaw uprawnień w konkretnym namespace. Definiuje co można robić i na jakich zasobach
- **ClusterRole** — jak Role ale działa w całym klastrze (przydatne dla monitoringu który czyta metryki ze wszystkich namespace)
- **RoleBinding / ClusterRoleBinding** — łączy ServiceAccount z Role. Bez tego SA nie ma żadnych uprawnień

Co zrobimy i weryfikacja

- ServiceAccount: aras-demo-sa w namespace default
- Role: get/list/watch na Podach i ConfigMapach — nic więcej
- RoleBinding: łączy aras-demo-sa z aras-demo-role

- Test: `kubectl auth can-i list pods --as=system:serviceaccount:default:aras-demo-sa` → yes
- Test: `kubectl auth can-i delete pods --as=system:serviceaccount:default:aras-demo-sa` → no

Błąd w pierwszym YAML: użyto `apiRef` zamiast `apiGroup` w polu `roleRef`. Kubernetes odrzucił obiekt jako nieprawidłowy. Rozwiązanie: poprawny `RoleBinding` w osobnym pliku z prawidłowym polem `apiGroup`: `rbac.authorization.k8s.io`. To jest nasz [ISSUE-011](#) (wewnętrzny).

Zastosowania produkcyjne

- CI/CD pipeline: osobny `ServiceAccount` z uprawnieniami tylko do `deploy` (`create/update Deployments`) bez możliwości czytania `Secrets`
- Monitoring: `Prometheus` potrzebuje `ClusterRole` z `get/list/watch` na wszystkich zasobach — ale bez uprawnień do modyfikacji
- Developer access: programista może robić `kubectl logs` i `kubectl exec` w "dev" namespace ale nie w "production"
- Audyt: wszystkie operacje RBAC są logowane — wiadomo kto co zrobił i kiedy

Silne strony

- Granularna kontrola: konkretne verby na konkretnych zasobach w konkretnych namespace
- Deklaratywna — Role i Bindings są w YAML, wersjonowane w Git
- Wbudowana w K8s — zero dodatkowych komponentów
- Audit logging — każda operacja jest zapisywana

Slabe strony / typowe problemy

- Skomplikowana konfiguracja dla dużych organizacji — setki ról i bindingów
- Brak "deny" rules — RBAC jest addytywny, nie można explicite zablokować
- Literówki w YAML powodują ciche błędy (`apiRef` zamiast `apiGroup` — nasz [ISSUE](#))
- Debugowanie: `kubectl auth can-i` to podstawowe narzędzie ale nie pokazuje dlaczego coś jest zablokowane